Solving the 15-Puzzle using Weighted A* Search

Saif Al Islam
2205119

July 6, 2026

1 Overview

This report presents an implementation of the Weighted A* search algorithm for solving the 15-puzzle problem. The implementation supports multiple heuristics and allows different weights to be applied to the heuristic function. The objective of this experiment is to observe how changing the weight affects solution quality, search cost, and execution time.

The evaluation uses fifteen solvable 4×4 puzzle instances.

2 Implementation

The implementation consists of the following major components.

- Puzzle representation
- Solvability checking using inversion parity
- State generation
- Priority Queue (Weighted A*)
- Multiple heuristic functions
- Benchmark framework

The evaluation function is

$$f(n) = g(n) + W \times h(n)$$

where

- $g(n)$ = path cost
- $h(n)$ = heuristic estimate
- W = heuristic weight

For this benchmark the Linear Conflict heuristic was used.

3 Experimental Setup

Fifteen scrambled but solvable 4×4 puzzle instances were tested.

Each puzzle was solved four times using

$$W = \{1.0,\ 1.2,\ 2.0,\ 5.0\}$$

The following metrics were collected:

- Solution length
- Nodes explored
- Nodes expanded
- Running time (milliseconds)

4 Experimental Results

Table 1: Benchmark Results

Puzzle	Weight	Steps	Explored	Expanded	Time(ms)
1	1.0	48	1953651	1028300	122016.955
1	1.2	48	360590	186613	23553.276
1	2.0	56	10678	5212	600.534
1	5.0	76	963	457	40.564
2	1.0	39	84103	43011	4578.127
2	1.2	39	8349	4192	496.391
2	2.0	45	1578	755	77.661
2	5.0	73	1205	586	54.241
3	1.0	33	9610	4844	363.809
3	1.2	33	3445	1700	121.230
3	2.0	39	282	135	10.022
3	5.0	51	506	234	20.953
4	1.0	47	1095708	583851	75534.390
4	1.2	47	340018	177179	22361.060
4	2.0	51	12564	6185	664.989
4	5.0	75	1010	483	40.149
5	1.0	39	7442	3716	262.744
5	1.2	39	2342	1144	77.470
5	2.0	39	441	206	12.837
5	5.0	47	417	191	10.628
6	1.0	33	3245	1586	87.738
6	1.2	33	819	396	36.374
6	2.0	35	417	194	17.736
6	5.0	43	187	85	7.775
7	1.0	40	107635	54700	5200.316

Puzzle	Weight	Steps	Explored	Expanded	Time(ms)
7	1.2	40	61271	30853	3328.958
7	2.0	46	1105	530	34.744
7	5.0	62	4483	2172	222.208
8	1.0	42	354237	182558	18644.232
8	1.2	44	134508	69102	7698.874
8	2.0	46	1713	830	79.345
8	5.0	86	3899	1896	181.171
9	1.0	40	91943	47179	4369.688
9	1.2	40	16517	8401	857.606
9	2.0	48	676	339	18.396
9	5.0	86	1526	744	51.790
10	1.0	36	13810	6953	569.532
10	1.2	36	2269	1122	98.817
10	2.0	36	3577	1745	155.512
10	5.0	66	4817	2364	214.725
11	1.0	39	57450	29586	2494.514
11	1.2	39	7638	3922	277.151
11	2.0	43	5613	2778	228.720
11	5.0	57	391	187	12.465
12	1.0	32	28726	14317	1208.932
12	1.2	32	12805	6290	638.588
12	2.0	34	2097	1000	83.501
12	5.0	54	6653	3234	275.214
13	1.0	41	4787	2444	165.072
13	1.2	43	1317	642	52.889
13	2.0	51	713	342	28.061
13	5.0	57	535	255	21.071
14	1.0	42	46237	24804	2132.575
14	1.2	42	18474	9666	951.671
14	2.0	46	11599	5710	560.363
14	5.0	80	1410	685	60.634
15	1.0	32	748	359	23.221
15	1.2	32	139	68	4.425
15	2.0	36	173	77	4.985
15	5.0	46	229	106	6.930

5 Analysis

The benchmark clearly demonstrates the trade-off introduced by Weighted A*.

- Increasing the heuristic weight significantly reduces the number of explored and expanded nodes.
- Runtime decreases dramatically as the search becomes more greedy.
- Larger weights generally produce longer (suboptimal) solutions.

- Weight 1.2 provides an excellent compromise between optimality and speed.
- Weight 5.0 produces the fastest searches but often at the cost of much longer solution paths.

6 Conclusion

The experimental results confirm the expected behavior of Weighted A*. Small heuristic weights preserve near-optimal solutions while substantially reducing search effort. Larger weights aggressively reduce runtime and memory usage but sacrifice solution optimality. Therefore, choosing the weight depends on whether optimality or execution speed is more important for the application.